UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

UNIDADE ACADÊMICA DE BELO JARDIM

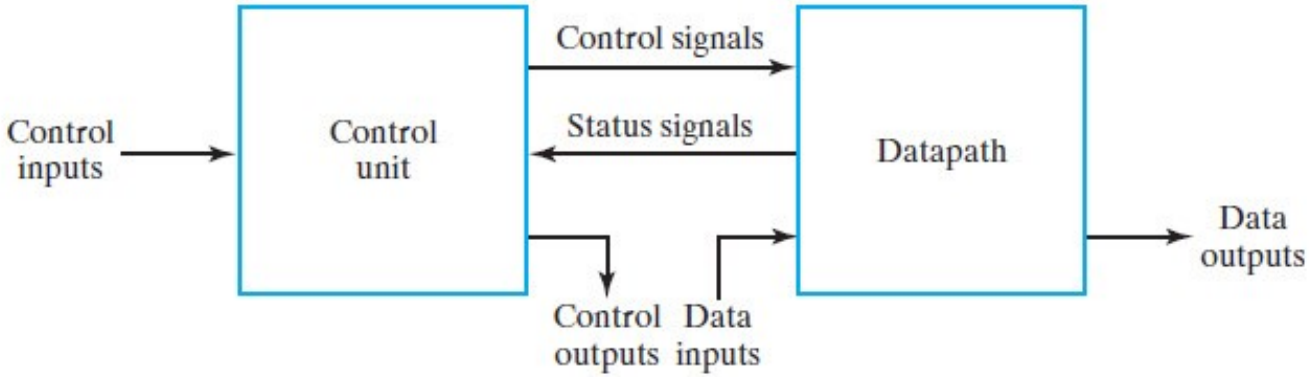
Sistemas Digitais:

Registradores

Prof. André Luís Silva Barbosa

Introdução

- Na maioria dos sistemas digitais, o sistema é particionado em dois módulos:
  - Um caminho de dados (datapath), no qual realiza o processamento das operações
  - Uma unidade de controle, que determina a sequência dessas operações



Introdução

- A unidade de controle:
  - Seleciona a operação
  - Determina a sequência dessas operações baseando-se no sinal de status e de entrada
- Datapath
  - Registradores, multiplexadores, registradores de deslocamento, circuitos e barramentos
  - Implementa microoperações
- Exemplos de operações nos registradores:
  - Load, clear, shift, count, etc.
- Uma operação elementar para armazenar dados no registrador é chamada **microoperação**

Introdução

- Exemplos de microoperações são:
  - Carregar o conteúdo de um registrador noutro;
  - Somar o conteúdo de dois registradores;
  - Incrementar o conteúdo de um registrador.

- Uma microoperação é, geralmente, mas não sempre, realizada em paralelo num vetor de bits durante um ciclo de clock.

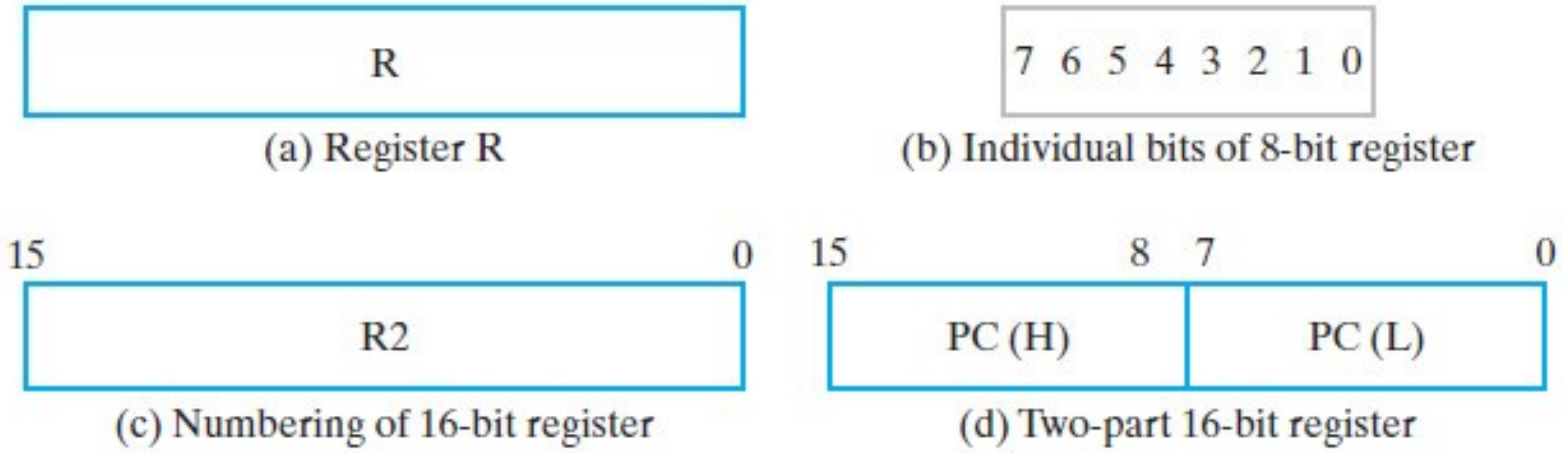
- O resultado de uma microoperação pode ser transferido para outro registrador, sobrescrevendo o conteúdo anterior.

Operações de transferência de registradores

- Um registrador que guarda um endereço da memória é usualmente chamado de registrador de endereço **AR**
- Outras designações para registradores são:
  - PC – Contador de programa (Program counter)
  - IR – Registrador de instruções (Instruction register)
  - R2 – Registrador 2

Operações de transferência de registradores

- A forma mais comum para representar registradores é com uma caixa retangular e o nome dentro.

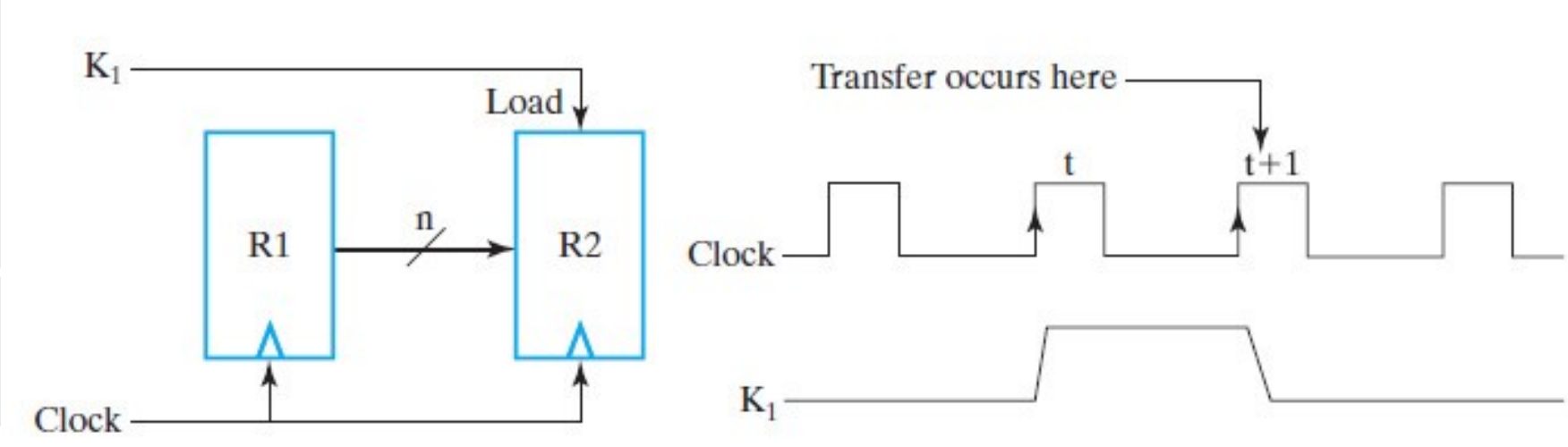


Operações de transferência de registradores

- O operador (←) é utilizado para representar a substituição de valores
  - $R2 \leftarrow R1$: Transfere o conteúdo de R1 para R2
  - R1 representa a fonte e R2 o destino
- Normalmente, não há a necessidade de realizar a transferência em cada pulso de clock, mas na dependência de um sinal de controle
- Uma declaração condicional pode ser realizada:
  - if $(K_1 = 1)$ then $(R2 \leftarrow R1)$
- Ou uma forma mais concisa
  - $K_1 : R2 \leftarrow R1$

Operações de transferência de registradores

- Transferência de R1 para R2 quando $K_1 = 1$



Operações de transferência de registradores

- Simbologia básica

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	$AR, R2, DR, IR$
Parentheses	Denotes a part of a register	$R2(1), R2(7{:}0), AR(L)$
Arrow	Denotes transfer of data	$R1 \leftarrow R2$
Comma	Separates simultaneous transfers	$R1 \leftarrow R2, R2 \leftarrow R1$
Square brackets	Specifies an address for memory	$DR \leftarrow M[AR]$

Operações de transferência de registradores

- RTL, VHDL e Verilog

Operation	Text RTL	VHDL	Verilog
Combinational assignment	=	<= (concurrent)	assign = (nonblocking)
Register transfer	←	<= (concurrent)	<= (nonblocking)
Addition	+	+	+
Subtraction	−	−	−
Bitwise AND	∧	and	&
Bitwise OR	∨	or	|
Bitwise XOR	⊕	xor	^
Bitwise NOT	– (overline)	not	~
Shift left (logical)	Sl	sll	<<
Shift right (logical)	Sr	srl	>>
Vectors/registers	*A*(3:0)	*A*(3 down to 0)	*A*[3:0]
Concatenation	‖	&	{ , }

Microoperações

- Transferência: transfere os bits de um registrador para outro
  - **Aritmética**: realiza uma operação aritmética nos dados dos registradores
  - **Lógica**: realiza uma manipulação entre bits nos dados dos registradores
  - **Descolamento**: desloca dados em registradores

Microoperações Aritméticas

- Adicionar, subtrair, incrementar e complementar
  - $R0 \leftarrow R1 + R2$ especifica a operação de soma
- A subtração implementa primeiro um complemento e depois adiciona.
  - $R0 \leftarrow R1 + \overline{R2} + 1$ especifica a operação de subtração por complemento de 2

Symbolic Designation	Description
$R0 \leftarrow R1 + R2$	Contents of $R1$ plus $R2$ transferred to $R0$
$R2 \leftarrow \overline{R2}$	Complement of the contents of $R2$ (1s complement)
$R2 \leftarrow \overline{R2} + 1$	2s complement of the contents of $R2$
$R0 \leftarrow R1 + \overline{R2} + 1$	$R1$ plus 2s complement of $R2$ transferred to $R0$ (subtraction)
$R1 \leftarrow R1 + 1$	Increment the contents of $R1$ (count up)
$R1 \leftarrow R1 - 1$	Decrement the contents of $R1$ (count down)

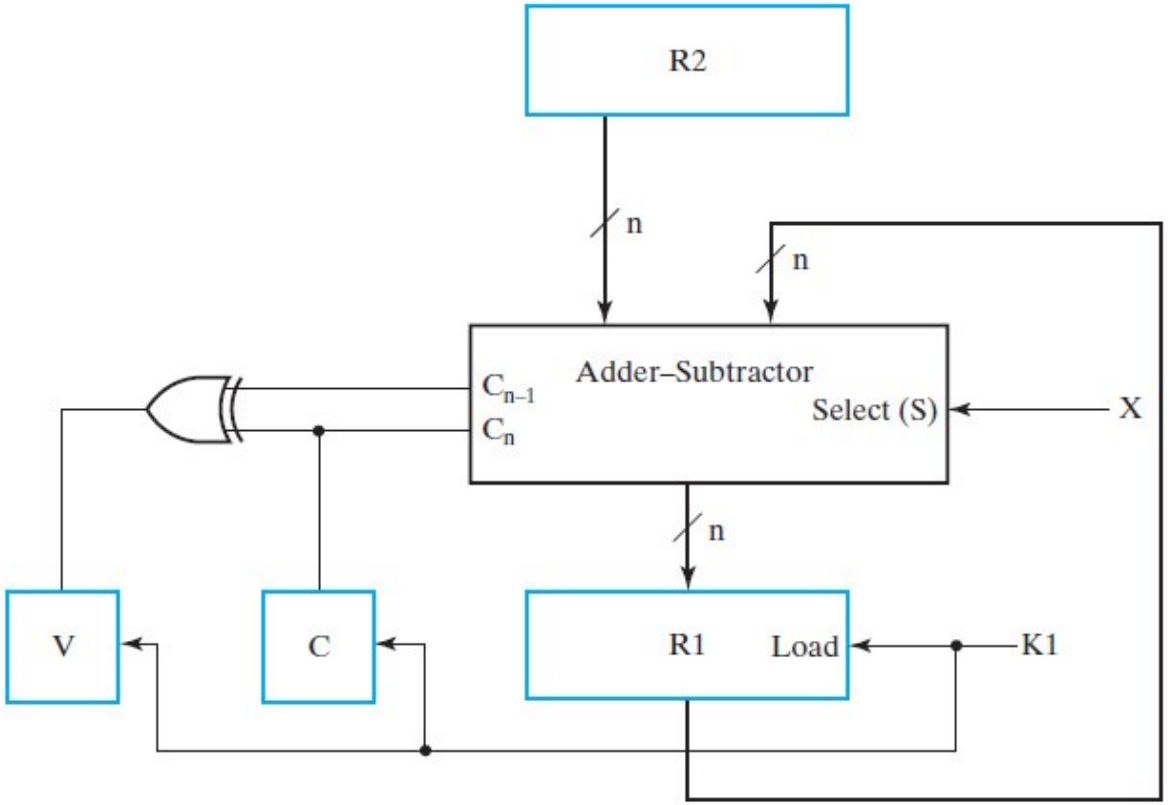
Microoperações Aritméticas

- As operações de multiplicação e divisão não estão representadas na tabela anterior;
- A multiplicação pode ser representada pelo símbolo * e a divisão por /
- Não são básicas porque são implementadas por uma sequência de microoperações
- Exemplo:
  - Há uma relação direta entre as declarações escritas nesta notação e as funções necessárias para sua implementação.
  - Considere:

$$\bar{X}K_1: R1 \leftarrow R1 + R2$$
$$XK_1: R1 \leftarrow R1 + \overline{R2} + 1$$

Microoperações Aritméticas

- A variável de controle K_1 ativa uma operação de soma ou subtração
- Se $K_1 = 1$ e ocorre no mesmo que a variável de controle X é igual a 0, então $\bar{X}K_1 = 1$ e o conteúdo de R2 é adicionado em R1
- Se $X = 1$ então $XK_1 = 1$ e o conteúdo R2 é subtraído do conteúdo de R1.
- Essa condição inibe que ambas sejam executadas simultaneamente.



Microoperações Aritméticas

- O bloco somador-subtrator recebe como entrada os dados de R1 e R2.
- A soma ou diferença é aplicada ao R1.
- A entrada S seleciona a operação no circuito.
  - Quando S=0 as duas entradas são somadas
  - Quando S=1 ocorre a subtração
- A saída do bloco principal é carregada em R1.

$$\bar{X}K_1 + XK_1 = (\bar{X} + X)K_1 = K_1$$

- Assim, a variável de controle X seleciona a operação e a variável K_1 carrega o resultado em R1
- O extrapolado é transferido para o FF V
- O carry do bit mais significativo é transferido para o FF C.

Sobre aritmética binária

- A tabuada da soma aritmética em:
  - 0 + 0 = 0
  - 0 + 1 = 1
  - 1 + 0 = 1
  - 1 + 1 = 0 (e “vai um” para o dígito de ordem superior)
  - 1 + 1 + 1 = 1 (e “vai um” para o dígito de ordem superior)

Sobre aritmética binária

- Exemplo: somar 011100_2 e 011010_2

$$
\begin{array}{rcccccc}
 & 0^{1} & 1^{1} & 1 & 1 & 0 & 0 \\
+ & 0 & 1 & 1 & 0 & 1 & 0 \\
\hline
 & 1 & 1 & 0 & 1 & 1 & 0
\end{array}
\qquad \leftarrow \text{“vai um”}
$$

Sobre aritmética binária

- Exemplo (subtração em complemento de 2): subtrair 26_{10} de 69_{10}

$$
\begin{array}{r}
01000101 \\
+\ 11100110 \\
\hline
00101011
\end{array}
\qquad
\begin{array}{l}
0011010 \\
\text{C1} \\
\text{:2)}
\end{array}
$$

Microoperações lógicas

- Operações que consideram cada bit no registrador separadamente e os tratam como variáveis binárias.

Symbolic Designation	Description
$R0 \leftarrow \overline{R1}$	Logical bitwise NOT (1s complement)
$R0 \leftarrow R1 \wedge R2$	Logical bitwise AND (clears bits)
$R0 \leftarrow R1 \vee R2$	Logical bitwise OR (sets bits)
$R0 \leftarrow R1 \oplus R2$	Logical bitwise XOR (complements bits)

Microoperações lógicas

- Considere a declaração abaixo:

$$(K_1 + K_2): R1 \leftarrow R2 + R3, R4 \leftarrow R5 \vee R6$$

Representa uma operação entre duas variáveis de controle.

- O operador + entre R2 e R3 especifica uma microoperação de soma.
- A operação OR está entre R5 e R6.
- É uma operação que pode ser facilmente implementada utilizando portas lógicas (circuitos combinacionais). Uma para cada bit.

Microoperações lógicas

- Microoperações lógicas podem:
  - Alterar valores de bits
  - Limpar um grupo de bits
  - Ou inserir novos valores num registrador

Microoperações lógicas

- Considere o exemplo a seguir para a microoperação lógica AND:

10101101 10101011	$R1$	(data)
00000000 11111111	$R2$	(mask)
00000000 10101011	$R1 \leftarrow R1 \wedge R2$	

- Para a microoperação OR

10101101 10101011	$R1$	(data)
11111111 00000000	$R2$	(mask)
11111111 10101011	$R1 \leftarrow R1 \vee R2$	

Microoperações lógicas

- Para a microoperação XOR

10101101 10101011	$R1$	(data)
11111111 00000000	$R2$	(mask)
01010010 10101011	$R1 \leftarrow R1 \oplus R2$	

Microoperações de deslocamento

- São usados para movimento lateral de dados.
- O conteúdo do registrador é deslocado para a direita ou esquerda
- Para a direita, significa em direção ao bit menos significativo
- Para a esquerda, significa em direção ao bit mais significativo

Microoperações de deslocamento

- A notação para esta microoperação é:

$$R0 \leftarrow sr\ R0, R1 \leftarrow sl\ R1$$

Type	Symbolic Designation	Eight-Bit Examples: Source $R2$	Eight-Bit Examples: After Shift: Destination $R1$
Shift left	$R1 \leftarrow \text{sl}\ R2$	10011110	00111100
Shift right	$R1 \leftarrow \text{sr}\ R2$	11100101	01110010

Microoperação em um único registrador

- Um único registrador pode servir de fonte para uma operação binária ou unária.
- A *lógica dedicada* é o nome dado ao circuito combinacional que implementa as microoperações.
- A *lógica compartilhada* é o nome dado para o circuito que implementa essa lógica para vários registradores.

Microoperação em um único registrador

- Transferência baseada em multiplexação
  - Considere a seguinte estrutura condicional:

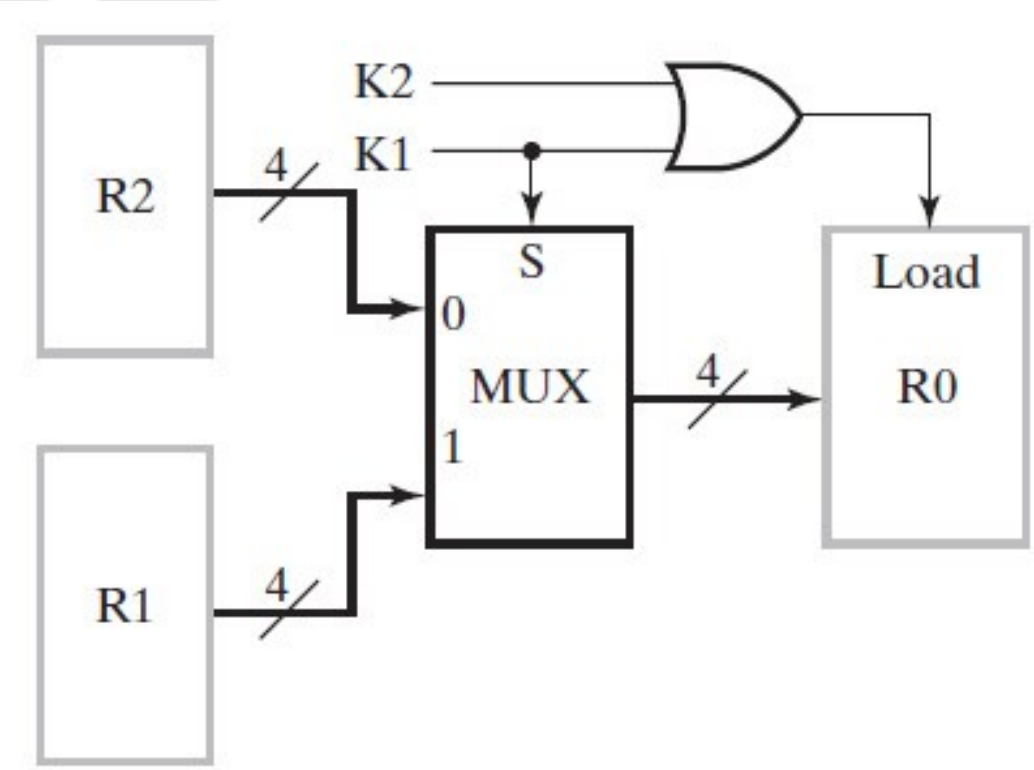
$$if\,(K_1 = 1)\ then\ (R0\ \leftarrow R1)\ else\ if\ (K_2 = 1)\ then\ (R0 \leftarrow R2)$$

  - Pode ser quebrado em:

$$K_1 : R0\ \leftarrow R1, \qquad \overline{K_1}K_2 : R0 \leftarrow R2$$

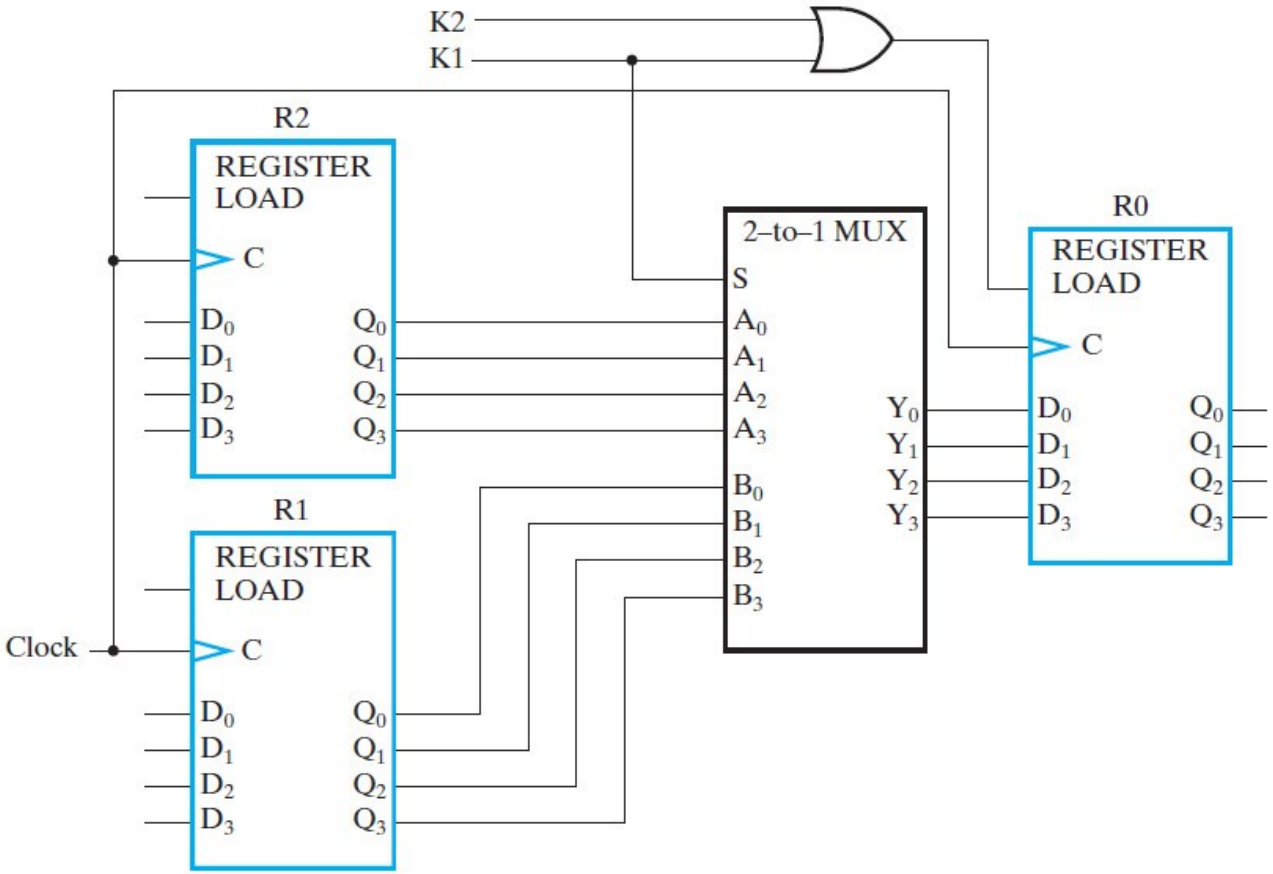
Microoperação em um único registrador

- Diagrama simplificado



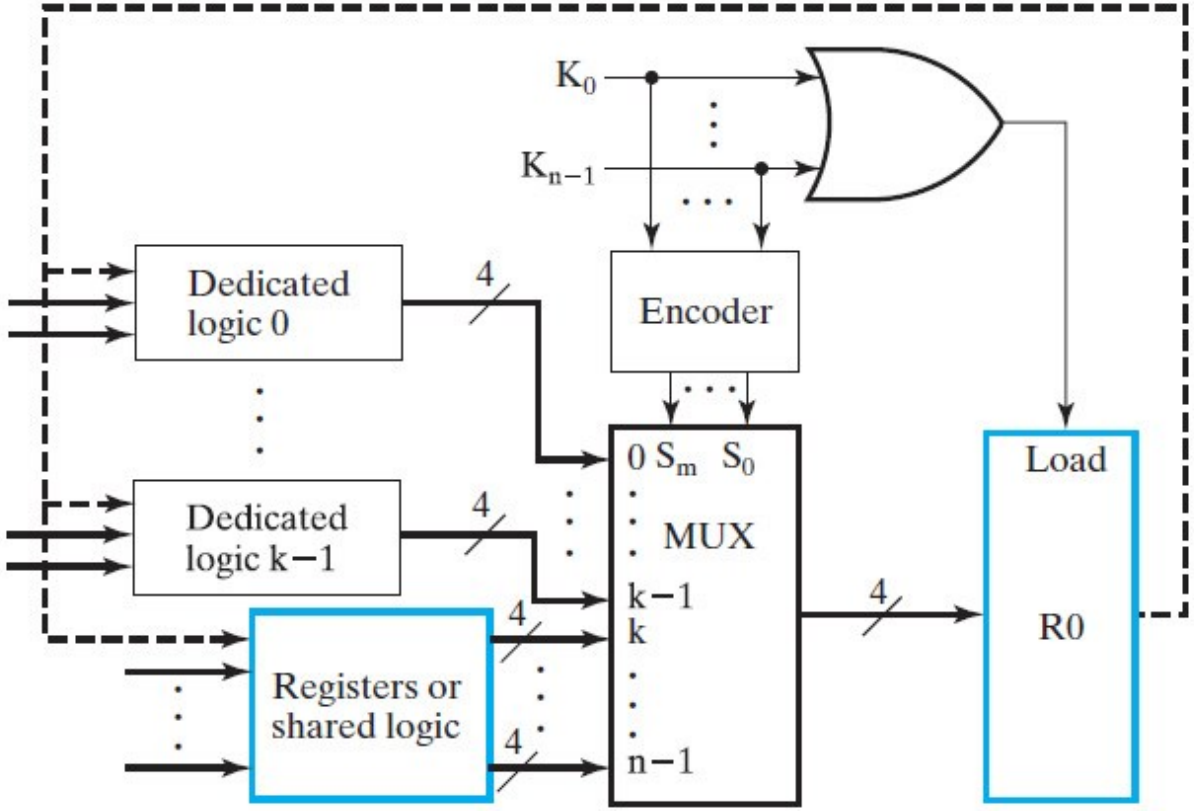
Microoperação em um único registrador

- Diagrama detalhado



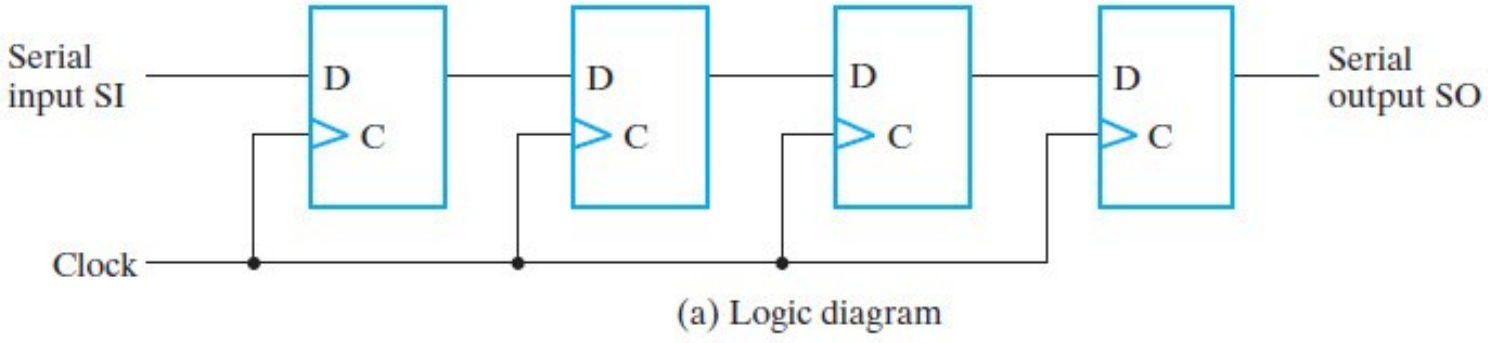
Microoperação em um único registrador

- Diagrama generalizado

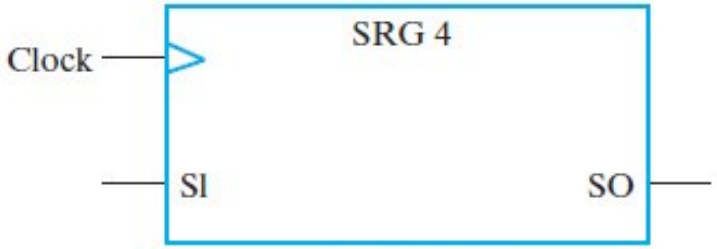


Registradores de deslocamento

- Capazes de deslocar bits em ambas as direções



(a) Logic diagram



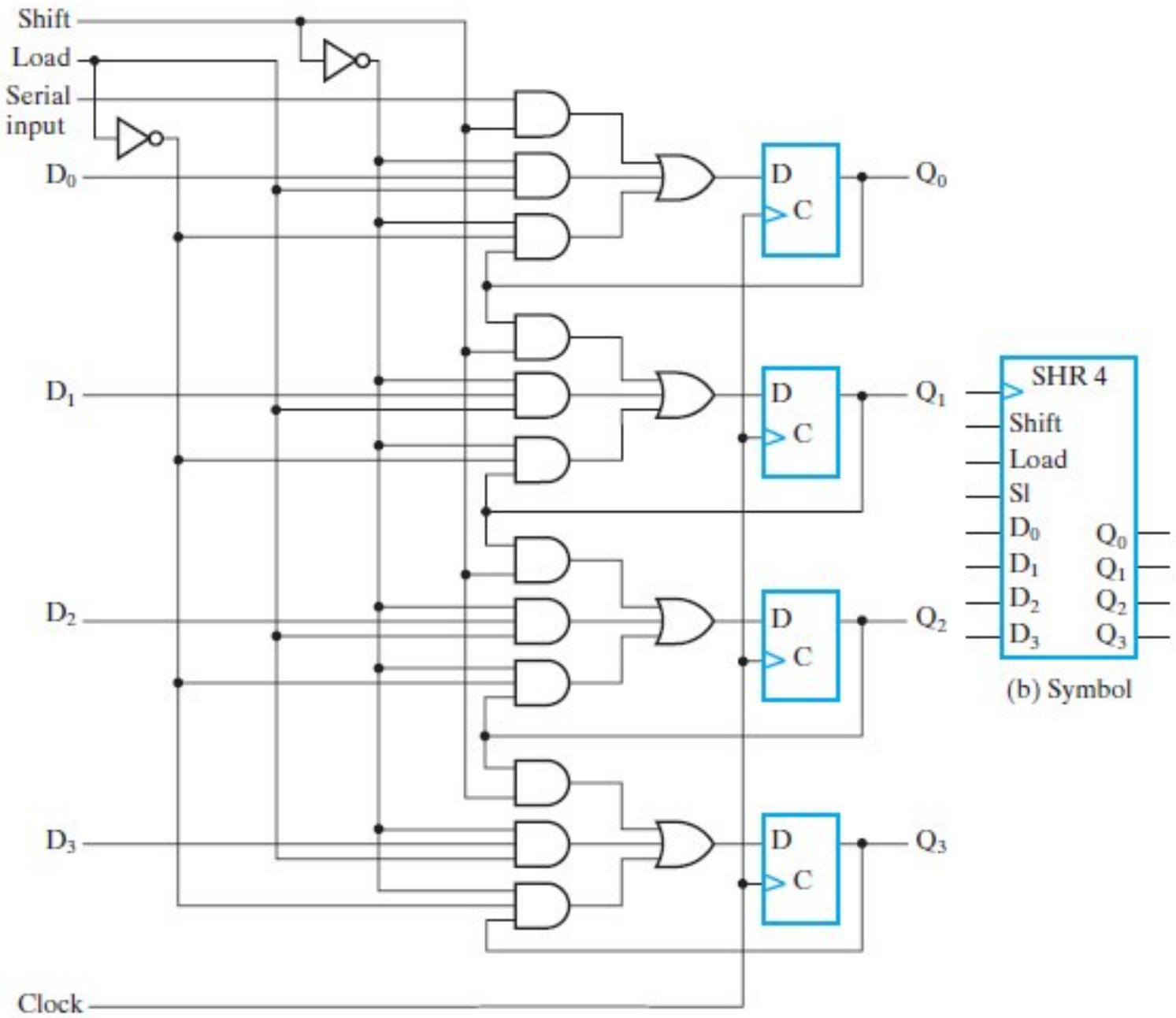
(b) Symbol

Carregamento em paralelo

$$Shift: Q \leftarrow \mathrm{sl}\, Q$$

$$\overline{Shift} \cdot Load: Q \leftarrow D$$

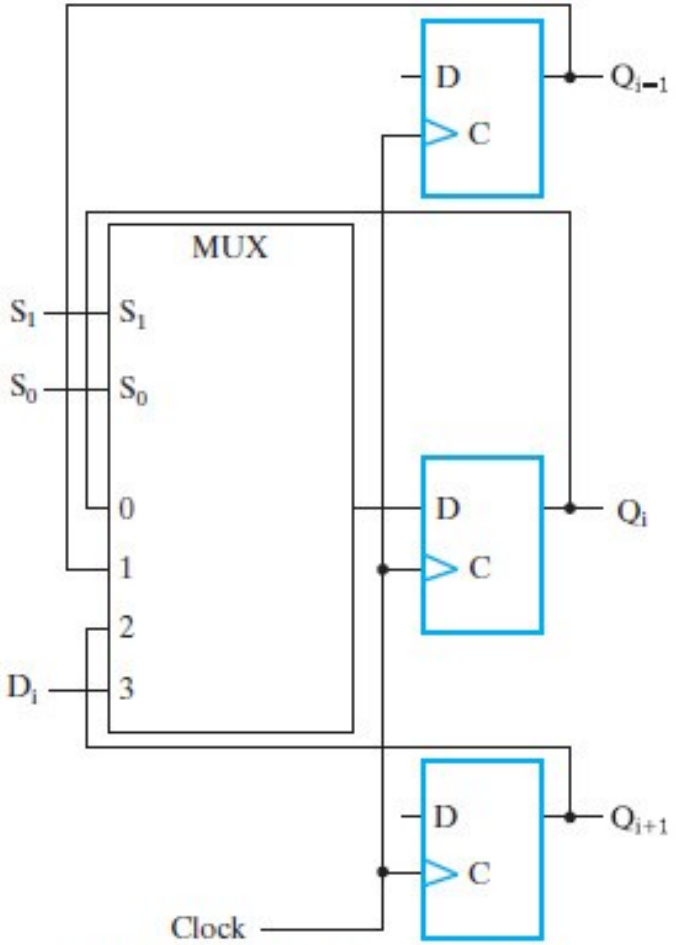
Shift	Load	Operation
0	0	No change (Hold)
0	1	Load parallel data
1	×	Shift left (down) from Q_0 to Q_3



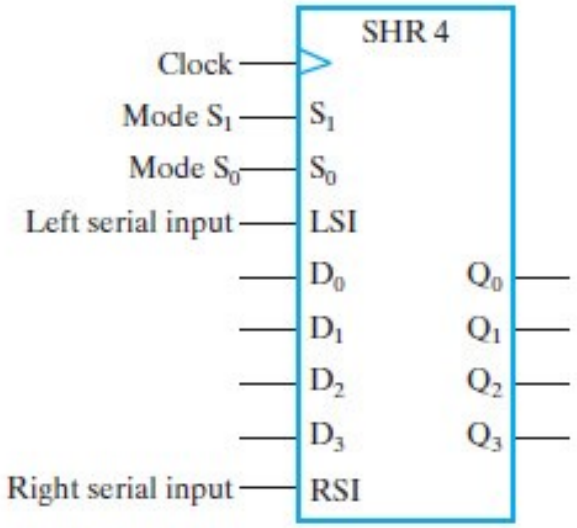
(b) Symbol

EX SEMINE SEGES

Registrador de deslocamento bidirecional



(a) Logic diagram of one typical stage



(b) Symbol

$$\overline{S}_1 \cdot S_0 \colon Q \leftarrow \mathrm{sl}\, Q$$

$$S_1 \cdot \overline{S}_0 \colon Q \leftarrow \mathrm{sr}\, Q$$

$$S_1 \cdot S_0 \colon Q \leftarrow D$$

Mode Control		Register Operation
S_1	S_0	
0	0	No change (Hold)
0	1	Shift left
1	0	Shift right
1	1	Parallel load

Bibliografia

- MANO, M. M. KIME, C. R., MARTIN, T. Logic and Computer Design Fundamentals. 5. ed: Pearson. 2015.

Obrigado!

andre.lsbarbosa@ufrpe.br

@andreluis.sb

@andreluissb